

PTB — Complete Topic Extraction & Priority List for UpMyRank

Part 1 — Every Topic Mentioned in the PDF

Optimization & Philosophy

- Generic chatbot optimizes for **user satisfaction** (instant answers, bypasses struggle)
- Educational bot optimizes for **learning gain** (desirable difficulty, deep mastery, critical thinking)
- Educational AI is a **high-stakes intervention** — wrong explanation = foundational failure
- LLM is a **composer**, not the source of knowledge — assembles Truth + Rules
- **The System > The Model** — architecture makes the AI correct, not the model itself

Core Responsibilities (4 roles)

- **Knowledge Guide** — explains concepts grounded strictly in course materials
- **Debugging Partner** — helps identify errors without providing the ultimate fix
- **Curriculum Navigator** — recommends next best problem based on live performance
- **Safety Guard** — refuses to solve graded assignments directly

Student Personas

- Define **who the system is for** before writing code — failure happens when assuming "Generic Learner"
- Personas as explicit **JSON state objects** that drive the personalization engine
- **Jordan (Beginner)**: high scaffolding, analogies, max 3 concepts per turn, ask confirmation, analogy-first
- **Taylor (Advanced)**: consultative mode, concise, direct critique, high information density, advanced references
- Each persona has: Background & Goals, System Requirements, Failure Mode, Policy, Success Metric
- Same concept explained **differently** per persona — not just tone, different depth and framing entirely

Student Model Schema

- Dynamic JSON object updated after every interaction

- Fields: `(student_id)`, `(persona)`, `(topic_mastery)` (per-topic float), `(preferred_style)`, `(common_misconceptions)` (list), `(allowed_hint_depth)`, `(last_session_summary)`
- Tracks **latent mastery** not just question history
- **Dynamic topic unlocks** — when mastery hits threshold, advanced topics unlock automatically
- Gives **private tutor feel** via persistent memory of weaknesses and strengths

Cognitive Load & Information Chunking

- Human brain processes limited new information at once
- Architecture must include a module that **monitors and restricts response length**
- Max **3 concepts per turn** — never dump everything at once
- **Pacing** — ask for student confirmation before moving to next sub-topic
- Prevents mental paralysis

Generic vs. Educational Response (the example)

- Generic: "Recursion is when a function calls itself. Here is a Fibonacci code block."
- Educational: analogy first ("like two mirrors facing each other"), then a Socratic question about application
- The **response style itself** is the differentiator — not just what is said but how

The Three Control Layers (Hierarchy)

- **Customization (Teacher Control / Stability)** — global rules, safety, pedagogy style. Set once per semester. Never changes mid-session.
- **Personalization (Student Control / Flexibility)** — individual difficulty, pacing, dynamic content. Updated per session.
- **Golden Dataset (Truth Control)** — expert-validated reference standard. Verifies factual accuracy.

Customization Implementation

- Implemented as a **Hidden Instruction Set** — a permanent filter governing the LLM
- Contains: role, grounding_source, strict_pedagogy_rules[], safety_fallback
- **Safety_fallback** re-states the core identity in case of instruction drift
- Defines what bot NEVER does ("Hard No's")

Personalization Layer

- **Persona-to-Response Pipeline:** Student Query → Personalization Layer (fetch student

model) → Policy Engine (select pedagogy style) → LLM Generation (inject constraints) → Final Response

- The persona is a **system variable** influencing token generation — not just a tone setting
- Policy Engine selects: scaffolding level, analogy use, information density, code length limit

Hinting vs. Leaking (Academic Integrity)

- Critical distinction: **conceptual scaffolding** vs. **solution leakage**
- Bot refuses direct answer but immediately offers conceptual hint or analogous problem
- Programmatic `safety_guardrail()` function: checks assignment blocklist, checks code generation length
- If code too long → force LLM to break into smaller chunks
- Ensures academic integrity without blocking learning

Misconception Detection Logic

- Include common **wrong mental models** in the truth layer
- If input matches a pattern → trigger **Correction Prompt** (correcting thinking, not just code)
- Pattern → Matched Misconception → Triggered Correction Action (3-column logic table)
- Examples: infinite loop pattern, shadowing built-in names, mutable default argument
- Moves from "correcting code" to "**correcting thinking**"

Golden Dataset

- Curated collection of structured **triplets**: Context + Question + Ideal Response
- Validated by **human experts** — not synthetically generated alone
- Pipeline: Collection → Synthetic Draft (Teacher LLM) → Human Validation → Git Versioning → Nightly Integration
- Used as absolute yardstick for **model selection and regression testing**
- Version controlled (e.g., `v1.2-Physics-Intro`) to track curriculum changes

Memory Boundaries (Selective Memory)

- DO remember: mastery level per syllabus topic
- DO remember: repeated misconceptions and syntax errors
- DO remember: preferred explanation depth (analogies vs. technical)
- DO NOT remember: unnecessary personal details or sensitive information
- DO NOT remember: unverified assumptions about student background

Failure Mode 1 — Retrieval Errors

- RAG fetches wrong concept (e.g., indexing instead of slicing)
- Detection: **low cosine similarity** between query and retrieved context
- Mitigation: self-correction or escalation flag
- Monitoring: log all "Low-Confidence" retrieval events

Failure Mode 2 — Pedagogy Drift

- LLM gives correct answer but **wrong pedagogy** (dumps answer instead of hinting)
- Detection: **Judge LLM** checks response against customization rubric
- Mitigation: guardrail layer blocks response, re-prompts LLM to follow Socratic rules
- Monitoring: track **Socratic Adherence Rate**

Failure Mode 3 — Latency & Flow

- Response > 3 seconds breaks student's **flow state**
- Detection: real-time latency tracking in API infrastructure
- Mitigation: **semantic caching** for common questions + streaming text
- Monitoring: alert if P99 latency exceeds 2 seconds

Evaluation — 4 Metric Categories

Technical Metrics:

- Correctness: output matches golden answer
- Faithfulness: response derived strictly from retrieved course notes
- Latency: under 2 seconds
- Retrieval Precision: fetched most relevant snippet

Pedagogical Metrics (Judge LLM):

- Scaffolding Quality: did the bot prompt thought?
- Misconception Correction: did the student fix their error?
- Rubric: score_0 (full answer) → score_1 (vague hint) → score_2 (leading Socratic question)

Learner Metrics:

- Time to Mastery: sessions to move from Beginner to Intermediate
- Reduced Repeated Errors: making same mistake less often
- Interaction Depth: asking more sophisticated questions over time

Safety Metrics:

- Refusal Accuracy: correctly refuses 100% of graded assignment queries
- Off-Topic Rejection: ignores irrelevant requests
- Cheating Detection Rate: jailbreak attempts blocked

Continuous Evaluation Loop

- Deploy → Collect Live Logs → Run through Judge LLM vs. Golden Dataset → Identify Failure Clusters → Update Dataset or System Prompt → Re-test → Deploy next version
- System is **never static** — improves daily

Regression Dashboard

- Live monitor: P99 latency, safety refusal rate, active sessions, avg accuracy, doc faithfulness
- If accuracy up but faithfulness down → **regression detected** → block deployment
- Prevents updates from degrading the system silently

Deployment Architecture

- Load Balancer, API Gateway, Vector DB, Monitoring Agent
- **Fallback Logic:** cached response or "I'm thinking" message if LLM slow

Part 2 — What UpMyRank Currently Has vs. Doesn't Have

PTB Concept	UpMyRank Status
Optimizes for learning gain (desirable difficulty)	✅ Socratic engine, forced attempt
Knowledge Guide (NCERT-grounded)	✅ RAG on 10,505 chunks
Debugging Partner (no ultimate fix)	✅ Hint escalation system
Curriculum Navigator (next best problem)	⚠️ Weak — practice mode is basic
Safety Guard (out-of-scope refusal)	✅ Out-of-scope intent classification
Student personas as JSON state objects	❌ No explicit persona schema
Jordan/Taylor style scaffolding levels	❌ No scaffolding level per student
Cognitive load control (max 3 concepts/turn)	❌ No response chunking module

PTB Concept	UpMyRank Status
Analogies in responses	✗ Not in prompts
Student confirmation before next concept	✗ Missing
Persona-to-Response Pipeline (Policy Engine)	⚠ Partial — mentor mode exists but no policy engine
Hinting vs. Leaking distinction	✓ Forced attempt gate
Misconception Detection Logic	✗ No misconception pattern table
Golden Dataset (triplets: context+question+ideal)	✗ No golden dataset
Memory Boundaries (what to remember/not)	✗ Not defined explicitly
Hidden Instruction Set (permanent filter)	✓ TUTOR_SYSTEM_PROMPT + safety_fallback partially
Customization vs. Personalization separation	✗ Mixed together in one prompt
Failure Mode 1: Retrieval error detection	✗ No cosine similarity threshold logging
Failure Mode 2: Pedagogy Drift (Judge LLM)	✗ No judge LLM
Failure Mode 3: Latency monitoring	✗ No latency tracking
Technical metrics (correctness, faithfulness)	⚠ SymPy verification only
Pedagogical metrics (scaffolding score)	✗ Not measured
Learner metrics (time to mastery, error reduction)	✗ Not tracked
Safety metrics (refusal accuracy)	✗ Not measured
Continuous Evaluation Loop	✗ No eval loop
Regression Dashboard	✗ Nothing
Semantic caching	✗ Redis exists but unused
Streaming text	✗ Not implemented

Part 3 — Priority List for UpMyRank System Redesign

Grouped into tiers. Each tier must be completed before moving to the next.

Tier 1 — Foundational (Do Before Any New Feature)

These are architectural gaps that affect everything else. Fixing them changes how the whole system behaves.

1. Separate Customization from Personalization in prompts Right now `TUTOR_SYSTEM_PROMPT` mixes global rules (Customization) with per-student context (Personalization). These need to be two separate prompt blocks. Customization is stable — set once. Personalization is dynamic — injected per student per session. This is the architectural prerequisite for everything else.

2. Define explicit Student Model Schema Add a formal `persona_profile` JSON to `student_memory`. Fields: `scaffolding_level` (HIGH/MEDIUM/LOW), `preferred_style` (analogy/formula/visual/example), `common_misconceptions[]`, `allowed_hint_depth`, `interaction_depth_score`. This drives the Policy Engine.

3. Build a Policy Engine Before LLM generation, run a Policy Engine step: read student model → select pedagogy_style → inject constraints into prompt. Currently the system goes student query → RAG → LLM. It needs to go: student query → student model fetch → policy selection → RAG → LLM with constraints. This is the Persona-to-Response Pipeline from the PDF.

4. Define Memory Boundaries explicitly Add a `MEMORY_SCOPE` constant to `prompts.py` listing what the AI remembers and doesn't. Inject into system prompt. Prevents LLM from making unverified assumptions about students.

Tier 2 — Response Quality (Do After Tier 1)

Directly improves what the student receives in every interaction.

5. Cognitive Load Chunking Add a response length constraint to `TUTOR_SYSTEM_PROMPT`: max 3 concepts per response. If a problem requires more, end with a check-in question ("Does that part make sense before we continue?"). This alone reduces student overwhelm significantly.

6. Misconception Detection Table Create `app/services/doubt/misconceptions.py` — a table of common JEE Physics wrong mental models (e.g., "centripetal force is a real force", "work done by normal force is always zero", "heavier objects fall faster"). Before generating a hint, check student's response against this table. If matched → trigger correction prompt targeting the thinking, not just the answer.

7. Analogy-first responses for low scaffolding students Add to Policy Engine: if `scaffolding_level == HIGH` → inject "Use a real-world analogy before any equation" into the prompt. If `scaffolding_level == LOW` → inject "Be concise, skip analogies, go to formalism directly." Same problem, fundamentally different response style.

8. Student confirmation before concept progression At hint_level 1 responses: end with "Does this make sense so far?" or "Can you tell me what that means in this problem?" before moving to

next hint. Currently hints just progress. This adds pacing.

Tier 3 — Truth & Evaluation (Do After Tier 2)

Makes the system measurable and improvable, not just functional.

9. Build a Golden Dataset (50 triplets to start) Create `data/golden_dataset.json` — 50 Physics problems with: context (NCERT chunk), question, ideal Socratic response at hint_level 0. These become the ground truth for regression testing. Start with the 10 most common topics in JEE Physics. Use GPT-4o to draft, then manually review each one.

10. Judge LLM for Pedagogy Drift detection After every LLM response at hint_level 0-2, run a fast Judge LLM call (gpt-4o-mini): "Score this response: 0 = gave full solution, 1 = vague hint, 2 = leading Socratic question." If score < 2 → log as pedagogy drift event. Don't block in real-time (latency cost) but use for daily analysis.

11. RAG retrieval confidence threshold Log cosine similarity of top retrieved chunk on every query. If max similarity < 0.5 → log as retrieval failure event. Surface to admin dashboard. Currently retrieval errors are silent — the LLM generates an answer regardless of retrieval quality.

12. Pedagogical metrics tracking Add to `session_events`: `scaffolding_score` (from Judge LLM), `misconception_detected` (bool), `hint_was_useful` (inferred from whether student solved after hint). These feed the learner metrics.

Tier 4 — Monitoring & Continuous Improvement (Post-Demo)

Makes the system self-improving over time.

13. Continuous Evaluation Loop (weekly batch job) Weekly script: pull all sessions from last 7 days → run each hint response through Judge LLM → identify failure clusters (topics where pedagogy drift rate > 20%) → generate report → update system prompts or golden dataset accordingly.

14. Latency tracking Log response time on every API call. Alert if P95 > 3 seconds. Currently there is no visibility into latency at all.

15. Regression Dashboard Simple dashboard (can be a Supabase query view): Socratic Adherence Rate, Avg Retrieval Similarity, Misconception Detection Rate, Out-of-Scope Refusal Rate, P95 Latency. Check before every deployment. Block deployment if any metric regresses > 5%.

16. Semantic caching (Redis) Cache responses for queries with cosine similarity > 0.95 to a prior query. Kills latency for repeated common questions. Redis is already in the stack — just unwired for this purpose.

The One Architectural Principle Worth Internalizing

The PDF's final message applied to UpMyRank:

The LLM (GPT-4.1-mini) is not UpMyRank's product. The system architecture — the Policy Engine, the Misconception Detection, the Golden Dataset, the Judge LLM, the Student Model Schema — is the product. Any competitor can swap in the same LLM. They cannot copy the system.

This is why Tier 1 matters more than any feature. Without the Policy Engine and Student Model Schema, UpMyRank is a well-prompted GPT wrapper. With them, it's a system.